- implicit `Deny` by default
- explicit `permits` required
- `forbids` always taking precedence

These rules ensure clear, predictable, and scalable policy evaluation that doesn't rely on policy order.

7.3.4 PARC as the bridge

In this main section, we've examined Cedar's PARC model in detail: the structure of a request (Principal, Action, Resource, and Conditions), how policies are written to match those elements, how the engine filters by scope, evaluates conditions, and ultimately makes decisions using deny-overrides.

PARC is more than just a conceptual model—it's the practical agreement Cedar expects in every request. The same four elements determine how you create policies, how those rules relate to real-world inputs, and how evaluation proceeds. This common language makes Cedar easy to work with, especially when updating legacy rules or collaborating with non-technical team members.

PARC acts as a bridge between theory and practice. In Chapter 8, when comparing Cedar to languages like OPA/Rego, OpenFGA, or XACML, PARC will serve as our common reference point. Each language addresses the question "Can this principal perform this action on this resource under these conditions?" However, they express it differently. Think of PARC as the foundation that helps us understand other policy languages, see where they align with PARC, and examine how their design choices influence your authorization strategy.

7.4 Cedar's unique design

Cedar isn't a general-purpose programming language—and that's intentional. It concentrates on one job: expressing authorization logic that is clear, correct, and efficient. Its design emphasizes safety over complexity, ease of analysis over expressiveness, and speed over generality. In this